

Informe de Laboratorio 07

Tema: SOAP Web services

INFORMACIÓN BÁSICA					
ASIGNATURA:	Sistemas Distribuidos				
TÍTULO DE LA PRÁCTICA:	<i>SOAP Web services</i>				
NÚMERO DE PRÁCTICA:	<i>07</i>	AÑO LECTIVO:	2026	NRO. SEMESTRE:	VII
FECHA DE PRESENTACIÓN	<i>29/05/2026</i>	HORA DE PRESENTACIÓN	12:00		
INTEGRANTE (s): • CHAMBILLA PERCA RICARDO • JUAN DIEGO GUTIERREZ CCAMA • RYAN FABIAN VALDIVIA SEGOVIA				NOTA:	
DOCENTE(s): <i>Mg. Maribel Molina Barriga</i>					

Repositorio del proyecto: https://github.com/rikich3/LAB_SISTEMAS_DISTRIBUIDOS.git

1. EJERCICIOS/PROBLEMAS PROPUESTOS

- **Ejercicio 1:** Servicio SOAP: Conversor de Temperatura (Celsius ↔ Fahrenheit) y Diseño de un Servicio de Ventas de Productos en Línea usando JAX-WS en Java.
- **Ejercicio 2:** Cliente SOAP con Python para consumir un servicio de calculadora existente utilizando la biblioteca `zeep`.
- **Actividad adicional (HTML + JS):** Consumidor interactivo de SOAP desde un navegador web y análisis de limitaciones.

2. Desarrollo de los Ejercicios

Toda la implementación del código y los componentes del laboratorio se encuentran organizados en el directorio `sesion_7/src/`.

2.1. Estructura del Proyecto en `sesion_7/src/`

```
1 sesion_7/src/  
2 |-- Ejercicio1/  
3 |   |-- pom.xml  
4 |   '-- src/  
5 |       '-- main/  
6 |           '-- java/  
7 |               '-- com/  
8 |                   '-- soap/  
9 |                       |-- ConversorSOAPInterface.java  
10 |                       |-- ConversorSOAP.java  
11 |                       |-- Producto.java  
12 |                       |-- VentaProductosSOAPInterface.java  
13 |                       |-- VentaProductosSOAP.java  
14 |                       |-- Publicador.java  
15 |                       '-- Consumidor.java  
16 |-- Ejercicio2/  
17 |   |-- client.py  
18 |   '-- requirements.txt  
19 '-- ActividadAdicional/  
20     '-- index.html
```

2.2. Ejercicio 1: Servidor y Cliente SOAP en Java

Este ejercicio incluye la definición de contratos para la conversión de temperatura, el sistema de inventario y venta de productos, el mecanismo de publicación con soporte CORS y el cliente de prueba en Java.

2.2.1. Conversor de Temperatura

Definimos la interfaz del servicio y su respectiva implementación lógica usando las especificaciones estándar de JAX-WS.

Listing 1: ConversorSOAPInterface.java (sesion_7/src/Ejercicio1/src/main/java/com/soap/ConversorSOAPInterface.java)

```
1 package com.soap;  
2  
3 import jakarta.jws.WebMethod;  
4 import jakarta.jws.WebService;  
5  
6 @WebService  
7 public interface ConversorSOAPInterface {  
8     @WebMethod  
9     double cToF(double c);  
10  
11     @WebMethod  
12     double fToC(double f);  
13 }
```

Listing 2: ConversorSOAP.java (sesion_7/src/Ejercicio1/src/main/java/com/soap/ConversorSOAP.java)

```
1 package com.soap;  
2  
3 import jakarta.jws.WebService;  
4
```

```
5 @WebService(endpointInterface = "com.soap.ConversorSOAPInterface")
6 public class ConversorSOAP implements ConversorSOAPInterface {
7     @Override
8     public double cToF(double c) {
9         return (c * 9.0 / 5.0) + 32.0;
10    }
11
12    @Override
13    public double fToC(double f) {
14        return (f - 32.0) * 5.0 / 9.0;
15    }
16 }
```

2.2.2. Servicio Avanzado: Ventas de Productos en Línea

Para la venta de productos en línea, primero definimos el modelo de datos **Producto** que requiere anotaciones JAXB para mapearse hacia XML. Posteriormente, diseñamos el servicio web con soporte para consultas y procesamiento concurrente de compras.

Listing 3: Producto.java (sesion_7/src/Ejercicio1/src/main/java/com/soap/Producto.java)

```
1 package com.soap;
2
3 import jakarta.xml.bind.annotation.XmlRootElement;
4 import java.io.Serializable;
5
6 @XmlRootElement
7 public class Producto implements Serializable {
8     private int id;
9     private String nombre;
10    private double precio;
11    private int stock;
12
13    public Producto() {}
14
15    public Producto(int id, String nombre, double precio, int stock) {
16        this.id = id;
17        this.nombre = nombre;
18        this.precio = precio;
19        this.stock = stock;
20    }
21
22    // Getters y Setters
23    public int getId() { return id; }
24    public void setId(int id) { this.id = id; }
25    public String getNombre() { return nombre; }
26    public void setNombre(String nombre) { this.nombre = nombre; }
27    public double getPrecio() { return precio; }
28    public void setPrecio(double precio) { this.precio = precio; }
29    public int getStock() { return stock; }
30    public void setStock(int stock) { this.stock = stock; }
31 }
```

Listing 4: VentaProductosSOAPInterface.java (sesion_7/src/Ejercicio1/src/main/java/com/soap/VentaProductosSOAPInterface.java)

```
1 package com.soap;
2
3 import jakarta.jws.WebMethod;
4 import jakarta.jws.WebService;
5 import java.util.List;
6
7 @WebService
8 public interface VentaProductosSOAPInterface {
9     @WebMethod
10     List<Producto> obtenerProductos();
11
12     @WebMethod
13     Producto obtenerProductoPorId(int id);
14
15     @WebMethod
16     String realizarVenta(int idProducto, int cantidad);
17 }
```

Listing 5: VentaProductosSOAP.java (sesion_7/src/Ejercicio1/src/main/java/com/soap/VentaProductosSOAP.java)

```
1 package com.soap;
2
3 import jakarta.jws.WebService;
4 import java.util.ArrayList;
5 import java.util.List;
6 import java.util.concurrent.ConcurrentHashMap;
7
8 @WebService(endpointInterface = "com.soap.VentaProductosSOAPInterface")
9 public class VentaProductosSOAP implements VentaProductosSOAPInterface {
10     private static final ConcurrentHashMap<Integer, Producto> inventario = new
11         ConcurrentHashMap<>();
12
13     static {
14         inventario.put(1, new Producto(1, "Laptop Gamer", 1200.00, 10));
15         inventario.put(2, new Producto(2, "Mouse Inalambrico", 25.50, 50));
16         inventario.put(3, new Producto(3, "Teclado Mecanico", 75.00, 30));
17     }
18
19     @Override
20     public List<Producto> obtenerProductos() {
21         return new ArrayList<>(inventario.values());
22     }
23
24     @Override
25     public Producto obtenerProductoPorId(int id) {
26         return inventario.get(id);
27     }
28
29     @Override
30     public synchronized String realizarVenta(int idProducto, int cantidad) {
31         Producto prod = inventario.get(idProducto);
32         if (prod == null) {
33             return "Error: Producto no encontrado.";
34         }
35         if (prod.getStock() < cantidad) {
36             return "Error: Stock insuficiente. Disponible: " + prod.getStock();
37         }
38     }
39 }
```

```
36     }
37     prod.setStock(prod.getStock() - cantidad);
38     double total = prod.getPrecio() * cantidad;
39     return "Venta Exitosa. Producto: " + prod.getNombre() + " | Cantidad: " + cantidad +
        " | Total: $" + total;
40 }
41 }
```

2.2.3. Publicador con Soporte CORS

Para permitir que las llamadas de AJAX provenientes de navegadores interactúen con el servidor SOAP, desarrollamos un filtro CORS que se adjunta al contexto HTTP antes de la publicación.

Listing 6: Publicador.java (sesion_7/src/Ejercicio1/src/main/java/com/soap/Publicador.java)

```
1 package com.soap;
2
3 import com.sun.net.httpserver.HttpExchange;
4 import com.sun.net.httpserver.HttpFilter;
5 import com.sun.net.httpserver.HttpServer;
6 import jakarta.xml.ws.Endpoint;
7 import java.io.IOException;
8 import java.net.InetSocketAddress;
9
10 public class Publicador {
11     public static void main(String[] args) {
12         try {
13             HttpServer server = HttpServer.create(new InetSocketAddress("localhost", 8081), 0);
14
15             // Filtro CORS personalizado
16             HttpFilter corsFilter = new HttpFilter() {
17                 @Override
18                 public void doFilter(HttpExchange exchange, Chain chain) throws IOException {
19                     exchange.getResponseHeaders().add("Access-Control-Allow-Origin", "*");
20                     exchange.getResponseHeaders().add("Access-Control-Allow-Methods", "POST,
21                         GET, OPTIONS");
22                     exchange.getResponseHeaders().add("Access-Control-Allow-Headers",
23                         "Content-Type, SOAPAction, Authorization");
24
25                     if ("OPTIONS".equalsIgnoreCase(exchange.getRequestMethod())) {
26                         exchange.sendResponseHeaders(204, -1);
27                         return;
28                     }
29                     chain.doFilter(exchange);
30                 }
31             };
32
33             // Creacion de contextos y aplicacion de filtro CORS
34             var contextCalc = server.createContext("/calculadora");
35             contextCalc.getFilters().add(corsFilter);
36             Endpoint endpointCalc = Endpoint.create(new ConversorSOAP());
37             endpointCalc.publish(contextCalc);
38
39             var contextVentas = server.createContext("/ventas");
40             contextVentas.getFilters().add(corsFilter);
41             Endpoint endpointVentas = Endpoint.create(new VentaProductosSOAP());
```

```
40         endpointVentas.publish(contextVentas);
41
42         server.start();
43         System.out.println("Servidor SOAP activo con soporte CORS en
44                             http://localhost:8081");
45         System.out.println("Conversor WSDL: http://localhost:8081/calculadora?wsdl");
46         System.out.println("Ventas WSDL: http://localhost:8081/ventas?wsdl");
47     } catch (Exception e) {
48         e.printStackTrace();
49     }
50 }
51 }
```

2.2.4. Consumidor en Java

A continuación se presenta el código para validar el consumo programático local desde un cliente en Java.

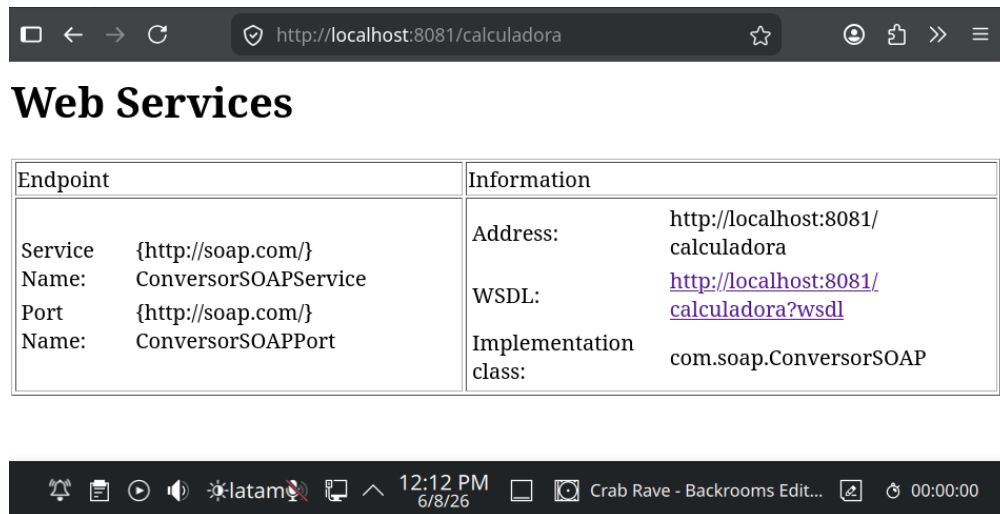
Listing 7: Consumidor.java (sesion_7/src/Ejercicio1/src/main/java/com/soap/Consumidor.java)

```
1 package com.soap;
2
3 import java.net.URL;
4 import javax.xml.namespace.QName;
5 import jakarta.xml.ws.Service;
6
7 public class Consumidor {
8     public static void main(String[] args) {
9         try {
10             // Consumo de servicio Conversor
11             URL urlCalc = new URL("http://localhost:8081/calculadora?wsdl");
12             QName qnameCalc = new QName("http://soap.com/", "ConversorSOAPService");
13             Service serviceCalc = Service.create(urlCalc, qnameCalc);
14             ConversorSOAPInterface calc = serviceCalc.getPort(ConversorSOAPInterface.class);
15
16             System.out.println("--- PRUEBA DE CONVERSOR SOAP ---");
17             System.out.println("30 grados Celsius a Fahrenheit: " + calc.cToF(30.0));
18             System.out.println("86 grados Fahrenheit a Celsius: " + calc.fToC(86.0));
19
20             // Consumo de servicio Ventas
21             URL urlVentas = new URL("http://localhost:8081/ventas?wsdl");
22             QName qnameVentas = new QName("http://soap.com/", "VentaProductosSOAPService");
23             Service serviceVentas = Service.create(urlVentas, qnameVentas);
24             VentaProductosSOAPInterface ventas =
25                 serviceVentas.getPort(VentaProductosSOAPInterface.class);
26
27             System.out.println("\n--- PRUEBA DE VENTAS SOAP ---");
28             System.out.println("Lista de Productos Disponibles:");
29             for (Producto p : ventas.obtenerProductos()) {
30                 System.out.println("ID: " + p.getId() + " | Nombre: " + p.getNombre() + " |
31                                     Stock: " + p.getStock() + " | Precio: $" + p.getPrecio());
32             }
33
34             System.out.println("\nRealizando transacciones...");
35             System.out.println(ventas.realizarVenta(1, 2)); // Compra exitosa
36         } catch (Exception e) {
37             e.printStackTrace();
38         }
39     }
40 }
```

```

34         System.out.println(ventas.realizarVenta(1, 15)); // Excede stock
35
36     } catch (Exception e) {
37         e.printStackTrace();
38     }
39 }
40 }

```



Endpoint		Information	
Service	{http://soap.com/}	Address:	http://localhost:8081/calculadora
Name:	ConversorSOAPService	WSDL:	http://localhost:8081/calculadora?wsdl
Port	{http://soap.com/}	Implementation class:	com.soap.ConversorSOAP
Name:	ConversorSOAPPort		

Figura 1: Captura de la solución del Ejercicio 1: servidor y cliente SOAP en Java.

2.3. Ejercicio 2: Cliente SOAP con Python

Utilizando la librería `zeep`, se consume de forma ágil el servicio de calculadora externo y público provisto por la guía de laboratorio.

Listing 8: client.py (sesion_7/src/Ejercicio2/client.py)

```

1 from zeep import Client
2
3 def main():
4     # URL de WSDL de calculadora de prueba
5     wsdl_url = 'http://www.dneonline.com/calculator.asmx?WSDL'
6     print(f"Estableciendo conexion con el servicio SOAP: {wsdl_url}...")
7
8     try {
9         client = Client(wsdl_url)
10        # Invocacion del metodo de suma expuesto por el Web Service
11        resultado = client.service.Add(5, 8)
12        print(f"Respuesta del servicio (Suma): 5 + 8 = {resultado}")
13    except Exception as e:
14        print(f"Error al consumir el servicio SOAP: {e}")
15
16 if __name__ == '__main__':
17     main()

```

```

12:15 sesion_7/src/Ejercicio2 main X? → python client.py
Conectando al servicio SOAP en: http://www.dneonline.com/calculator.asmx?WSDL...
Invocando Add(5, 8)...

=== Resultado Esperado: 13 ===
Resultado Obtenido: 13

12:16 sesion_7/src/Ejercicio2 main X? →

```

Figura 2: Captura de la solución del Ejercicio 2: cliente SOAP en Python.

2.4. Actividad Adicional: Consumo de SOAP desde Navegador y Limitaciones

Se implementó un dashboard en HTML y JavaScript que construye de forma dinámica el sobre (Envelope) XML y parsea la respuesta XML usando el estándar del navegador.

Listing 9: index.html (sesion_7/src/ActividadAdicional/index.html)

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <title>SOAP Client Dashboard</title>
6   <style>
7     body { font-family: Arial, sans-serif; background-color: #1e1e2e; color: #cdd6f4;
8       margin: 30px; }
9     .container { max-width: 800px; margin: auto; }
10    .card { background-color: #313244; padding: 20px; border-radius: 8px; margin-bottom:
11      20px; box-shadow: 0 4px 6px rgba(0,0,0,0.3); }
12    h2 { border-bottom: 2px solid #f5c2e7; padding-bottom: 8px; color: #f5c2e7; }
13    input, button { padding: 10px; margin: 5px 0; border-radius: 4px; border: none; }
14    input { width: 100%; box-sizing: border-box; background-color: #45475a; color:
15      #cdd6f4; }
16    button { background-color: #a6e3a1; color: #11111b; cursor: pointer; font-weight:
17      bold; }
18    button:hover { background-color: #94e2d5; }
19    .result { padding: 10px; background-color: #181825; border-left: 4px solid #fab387;
20      margin-top: 10px; }
21  </style>
22 </head>
23 <body>
24   <div class="container">
25     <h1>Dashboard Interactivo de Servicios SOAP</h1>
26
27     <div class="card">
28       <h2>1. Conversor de Temperatura (Celsius a Fahrenheit)</h2>
29       <label for="celsius">Temperatura en Celsius:</label>
30       <input type="number" id="celsius" value="30">
31       <button onclick="convertirCelsius()">Convertir</button>
32       <div class="result" id="res-conversor">Resultado: Esperando entrada...</div>
33     </div>
34
35     <div class="card">
36       <h2>2. Sistema de Ventas (Comprar Producto)</h2>

```

```

32     <label for="prod-id">ID de Producto:</label>
33     <input type="number" id="prod-id" value="1">
34     <label for="cantidad">Cantidad:</label>
35     <input type="number" id="cantidad" value="2">
36     <button onclick="realizarVenta()">Ejecutar Compra</button>
37     <div class="result" id="res-ventas">Resultado: Esperando transaccion...</div>
38   </div>
39 </div>
40
41 <script>
42   async function enviarSoap(url, soapAction, envelope, extractorXPath, idOutput) {
43     try {
44       const response = await fetch(url, {
45         method: 'POST',
46         headers: {
47           'Content-Type': 'text/xml; charset=utf-8',
48           'SOAPAction': soapAction
49         },
50         body: envelope
51       });
52
53       if (!response.ok) {
54         throw new Error('HTTP Error ${response.status}');
55       }
56
57       const xmlText = await response.text();
58       const parser = new DOMParser();
59       const xmlDoc = parser.parseFromString(xmlText, "text/xml");
60
61       // Extraccion basica del resultado mediante selectores
62       const elem = xmlDoc.getElementsByTagName(extractorXPath)[0];
63       document.getElementById(idOutput).innerText = elem ? "Respuesta: " +
64         elem.textContent : "Error al procesar XML de respuesta.";
65     } catch (err) {
66       document.getElementById(idOutput).innerText = "Error: " + err.message;
67     }
68   }
69
70   function convertirCelsius() {
71     const celsius = document.getElementById('celsius').value;
72     const envelope = '<?xml version="1.0" encoding="utf-8"?>
73     <soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
74       xmlns:soap="http://soap.com/">
75       <soapenv:Body>
76         <soap:cToF><arg0>${celsius}</arg0></soap:cToF>
77       </soapenv:Body>
78     </soapenv:Envelope>';
79     enviarSoap('http://localhost:8081/calculadora', '', envelope, 'return',
80       'res-conversor');
81   }
82
83   function realizarVenta() {
84     const id = document.getElementById('prod-id').value;
85     const cant = document.getElementById('cantidad').value;
86     const envelope = '<?xml version="1.0" encoding="utf-8"?>
87     <soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"

```

```

85     xmlns:soap="http://soap.com/"
86     <soapenv:Body>
87         <soap:realizarVenta>
88             <arg0>${id}</arg0>
89             <arg1>${cant}</arg1>
90         </soap:realizarVenta>
91     </soapenv:Body>
92 </soapenv:Envelope>;
93     enviarSoap('http://localhost:8081/ventas', '', envelope, 'return', 'res-ventas');
94 }
95 </script>
96 </body>
</html>

```



Figura 3: Captura de la interfaz de la actividad adicional.

2.4.1. Análisis de Limitaciones de SOAP en el Navegador

A partir de la experimentación y el diseño del frontend para consumir los endpoints locales, se documentaron las siguientes limitaciones estructurales:

1. **CORS (Cross-Origin Resource Sharing):** Los servidores SOAP legados rara vez están configurados para responder de manera nativa a solicitudes de origen cruzado. Al realizar peticiones desde un navegador web, el mecanismo de pre-vuelo HTTP OPTIONS fallará a menos de que se implemente una lógica explícita en el servidor de destino para inyectar los encabezados de control.
2. **Payload XML Complejo:** SOAP requiere encapsular todos los parámetros dentro de una estructura jerárquica de XML (Envelope, Body, Namespaces, etc.). Construir estas plantillas manualmente en JS de lado cliente es ineficiente y propenso a errores tipográficos, a diferencia de REST que utiliza la serialización nativa de JSON.

3. **Procesamiento e Ineficiencia del Parser:** El navegador se ve forzado a instanciar la clase `DOMParser()` para transformar los flujos de datos recibidos y ejecutar consultas sobre etiquetas para extraer un único valor primitivo, consumiendo memoria y ciclos de CPU innecesarios.
4. **Seguridad de Credenciales:** Los esquemas robustos basados en **WS-Security** (por ejemplo, firmas de XML o tokens) no pueden ser implementados de forma segura en un entorno Web abierto a inspección del lado cliente, pues expone los secretos del negocio directamente al usuario final.

2.5. Instrucciones para Ejecutar los Componentes

1. **Levantar el Servidor Java (SOAP):** Navegue a la carpeta `sesion_7/src/Ejercicio1/`, recompile el proyecto Maven y ejecute el publicador:

```
1 mvn clean compile
2 mvn exec:java -Dexec.mainClass="com.soap.Publicador"
```

2. **Correr Cliente Java Consola:** En una terminal paralela bajo el mismo directorio, ejecute:

```
1 mvn exec:java -Dexec.mainClass="com.soap.Consumidor"
```

3. **Ejecutar el Cliente Python:** Ingrese a `sesion_7/src/Ejercicio2/`, instale las dependencias especificadas y corra el script:

```
1 pip install -r requirements.txt
2 python client.py
```

4. **Abrir Interfaz Gráfica:** Abra directamente en su navegador el archivo `sesion_7/src/ActividadAdicional/index`. Se podrán verificar los consumos con peticiones asíncronas en vivo al servidor Java que se encuentra activo en el puerto 8081.

3. Conclusiones

- Se implementó de manera exitosa la publicación y el consumo de servicios web usando SOAP en entornos heterogéneos. JAX-WS proporciona herramientas adecuadas en Java para estructurar contratos robustos gracias a las anotaciones JAXB.
- El uso de clientes dinámicos en Python mediante **zeep** simplifica drásticamente el proceso de interpretación del esquema WSDL en tiempo de ejecución, eliminando la necesidad de precompilar clases locales de stub.
- A pesar de la robustez de SOAP para escenarios corporativos estrictos, su uso en interfaces web modernas presenta serias desventajas de latencia, legibilidad y seguridad en el navegador debido a la rigidez del protocolo y la complejidad inherente de los esquemas XML.

4. CUESTIONARIO

1. ¿Puedo utilizar un componente JavaBeans para implementar un servicio web utilizando la invocación de SOAP sobre JMS (Java Message Service)?

Sí, es completamente viable. En la arquitectura Java Enterprise (EE/Jakarta), se pueden exponer clases del tipo JavaBeans o de sesión EJB (Message-Driven Beans) como puntos de enlace (endpoints) de servicios web que utilicen JMS como canal de transporte subyacente. Para ello se define la pila JAX-WS especificando una dirección URI con esquema `jms://` en lugar de la clásica `http://`. El contenedor se encarga de recibir los mensajes JMS conteniendo los payloads SOAP, derivándolos al Bean correspondiente.

2. ¿Cómo funciona la mensajería bidireccional con la implementación de SOAP y JMS? ¿Da soporte a varios clientes realizando solicitudes simultáneas?

La comunicación asíncrona bidireccional sobre JMS se establece bajo el patrón de diseño Request-Reply. El cliente genera un mensaje SOAP sobre una cola de solicitudes (*Request Queue*) e introduce en la cabecera JMS el campo `JMSReplyTo` indicando la cola de respuesta correspondiente, junto a un identificador único en `JMSCorrelationID`.

El servidor procesa el requerimiento de forma asíncrona y coloca la respuesta dentro de la cola indicada utilizando el mismo identificador de correlación. Sí soporta múltiples solicitudes simultáneas gracias a que el middleware o agente de mensajería (Message Broker) administra colas con balanceo de carga y multihilo para atender concurrentemente el despacho de respuestas mediante subprocesos aislados.

3. ¿Por qué SOAP sigue siendo utilizado en sistemas empresariales críticos pese al auge de REST?

SOAP conserva una gran relevancia en infraestructuras transaccionales corporativas complejas debido a las siguientes características esenciales:

- **Seguridad Robusta (WS-Security):** Permite el cifrado y la firma digital por bloques de los datos XML a nivel de mensaje, de modo que el payload se mantiene protegido inclusive si es ruteado por servidores intermediarios no seguros.
- **Transaccionalidad (WS-Coordination / WS-AtomicTransaction):** Garantiza consistencia transaccional (ACID) distribuida sobre múltiples sistemas participantes.
- **Contrato Estricto (WSDL):** Define esquemas inalterables para la validación automática de datos en ambos extremos, crucial para evitar inconsistencias de formato en integraciones bancarias e industriales.

4. ¿Qué implicancias tiene el uso de XML en el rendimiento de sistemas distribuidos de alta concurrencia?

El uso de XML introduce importantes penalizaciones sobre el desempeño del sistema:

- **Sobrecarga de Red (Payload Size):** XML añade una cantidad significativa de metadatos repetitivos mediante etiquetas de apertura y cierre, lo que genera un tamaño de paquete sustancialmente superior a alternativas compactas como JSON o Protocol Buffers.
- **Uso Intensivo de CPU y Memoria:** Los analizadores sintácticos de XML (como DOM o SAX) requieren ciclos de reloj adicionales para mapear la jerarquía de nodos en memoria, generando además una alta carga en el Recolector de Basura (Garbage Collector) debido a la creación masiva de objetos de ciclo de vida corto.

5. ¿En qué escenarios arquitectónicos SOAP resulta más adecuado que REST? Justifique técnicamente.

SOAP resulta preferible en escenarios caracterizados por los siguientes requisitos:

- **Sistemas Financieros y Bancarios Transaccionales:** Donde la pérdida o duplicación parcial de una operación sobre un clúster distribuido puede comprometer estados de cuenta. El uso de las especificaciones *WS-AtomicTransaction* previene la ejecución de transacciones corruptas.
- **Canales de Transporte Heterogéneos:** En arquitecturas donde el envío del mensaje no está restringido estrictamente a HTTP. SOAP puede operar sobre colas JMS, SMTP o TCP puros sin perder los atributos de control e integridad definidos en su Envelope XML.
- **Estándares de Firma Extremo a Extremo:** Casos de uso de autenticación de alta seguridad donde el servidor intermedio procesa parte de la carga útil sin necesidad de descifrar la información confidencial contenida en otras partes del sobre SOAP.

5. Referencias

- Tanenbaum, A.S. (2008). *Sistemas distribuidos: principios y paradigmas*. México. Pearson Educación.
- Oracle JAX-WS Documentation (Jakarta XML Web Services). Recuperado de: <https://eclipse-ee4j.github.io/metro-jax-ws/>
- Zeep: Python SOAP Client Documentation. Recuperado de: <https://docs.python-zeep.org/en/master/>